

Computer Organization and Architecture

Complete MCQ-Oriented Notes

IBPS SO (IT OFFICER)
COMPLETE PREPARATION SERIES

100% SYLLABUS COVERAGE
CONCEPT-FOCUSED LEARNING
MCQ-ORIENTED APPROACH
QUICK REVISION READY

COMPILED BY
Asabhya Maanav

2026 EDITION
VERSION 1.0

Table of Contents

Computer Organization and Architecture — Complete Notes	4
1. Number Systems and Arithmetic	4
1.1 Number Systems	4
1.2 Binary Arithmetic	4
1.3 Data Representation	5
2. Codes	6
2.1 Binary Codes	6
2.2 Error Detection Codes	7
3. Digital Logic (Foundation)	8
3.1 Logic Gates	8
3.2 Boolean Algebra and Laws	8
3.3 Karnaugh Map (K-Map) Simplification	8
3.4 Combinational Circuits	8
3.5 Sequential Circuits	8
4. Instruction Set and Addressing	9
4.1 Machine Instructions	9
4.2 Addressing Modes	9
4.3 Program Control	10
5. CPU Organization	10
5.1 CPU Structure	10
5.2 Register Organization	10
5.3 Control Unit	11
5.4 Arithmetic Logic Unit (ALU)	11
6. Pipelining and Parallelism	11
6.1 Instruction Pipelining Concepts	11
6.2 Pipeline Stages and Speedup	11
6.3 Pipeline Hazards	12
6.4 RISC vs CISC	12
6.5 Parallel Processing — Flynn's Classification	12
7. Memory Organization	13
7.1 Memory Hierarchy	13
7.2 Memory Types	13
7.3 Cache Memory	13
7.4 Virtual Memory and Memory Management	14
8. Input-Output Organization	14
8.1 I/O Interface and Bus	14

8.2 Modes of Data Transfer	14
8.3 Interrupts	15
8.4 I/O Processor (IOP)	15
9. Quick Revision and Exam Pointers	15

Computer Organization and Architecture — Complete Notes

This guide builds Computer Organization & Architecture from first principles to advanced exam level for the **IBPS SO (IT Officer)** examination. Every unit follows the official syllabus: number systems, codes, digital logic, instruction sets, CPU organization, pipelining, memory, and I/O.

Memory Trick: Sequence the whole subject as "**NC-DI-CP-MI**" — **N**umbers, **C**odes, **D**igital logic, **I**nstructions, **C**PU, **P**ipelining, **M**emory, **I**/O.

1. Number Systems and Arithmetic

1.1 Number Systems

A **number system** (or positional system) represents numbers using a fixed set of digits and a **base/radix (r)**. The value of a digit depends on its position (weight = r^{position}).

Number System	Base (Radix)	Valid Digits	Example
Binary	2	0, 1	1011
Octal	8	0–7	745
Decimal	10	0–9	925
Hexadecimal	16	0–9, A–F	2AF

1.1.1 Positional Value (Weighted Expansion)

The decimal value of any number = sum of (digit × base^{position}). Positions left of the radix point are **0, 1, 2, ...**; positions right are **-1, -2, -3, ...**.

Binary 1011.1 to Decimal:
 $1 \times 2^3 + 0 \times 2^2 + 1 \times 2^1 + 1 \times 2^0 + 1 \times 2^{-1}$
 $= 8 + 0 + 2 + 1 + 0.5 = 11.5$ (decimal)

1.1.2 Base Conversions (All Directions)

- **Any base to Decimal:** use positional weighted expansion (multiply each digit by base^{position}).
- **Decimal integer to any base:** repeated **division** by base; remainders read **bottom-to-top**.
- **Decimal fraction to any base:** repeated **multiplication** by base; carries read **top-to-bottom**.
- **Binary to Octal:** group bits in **3s** from the radix point.
- **Binary to Hex:** group bits in **4s** from the radix point.
- **Octal/Hex to Binary:** expand each digit into **3 / 4** bits respectively.

Shortcut: 1 hex digit = **4 bits** (a nibble); 1 octal digit = **3 bits**; 1 byte = **2 hex digits**. Hex is just binary read 4 bits at a time.

MCQ Trap: To go binary→octal use groups of **3**, but binary→hex use groups of **4**. Pad the leftmost group with leading zeros if incomplete; for fractions, pad the rightmost group with trailing zeros.

1.2 Binary Arithmetic

1.2.1 Binary Addition and Subtraction

Binary addition rules: **0+0=0**, **0+1=1**, **1+0=1**, **1+1=10** (write 0, carry 1), **1+1+1=11** (write 1, carry 1). Binary subtraction borrows **2** (not 10) from the next higher position.

1.2.2 Binary Multiplication and Division

Multiplication uses **shift-and-add** of partial products (rules: **0×0=0**, **1×1=1**). Division uses **shift-and-subtract** like long division. Booth's algorithm is an efficient signed multiplication method using **2's complement**.

1.2.3 Complements: 1's and 2's

For base r there are two complements: the **$(r-1)$'s complement** (diminished radix) and the **r 's complement**.

Complement	Rule	Example (1010)
1's complement	Invert (flip) every bit	0101
2's complement	1's complement + 1, OR copy bits up to & including first 1 from LSB then invert the rest	0110
9's complement (decimal)	Subtract each digit from 9	—
10's complement (decimal)	9's complement + 1	—

Memory Trick: "Invert, then +1" gives 2's complement. The fast trick: scan from the **right (LSB)**, copy bits up to and including the first 1, then flip everything to the left.

1.2.4 Subtraction Using Complements

To compute $A - B$ in r 's (2's) complement: add A to the **2's complement of B** . If there is an **end carry**, discard it (result is positive). If there is **no end carry**, the result is negative and is in 2's-complement form. In **1's complement** subtraction, an end carry must be **added back** (end-around carry).

$A - B$ using 2's complement ($A=0110=6$, $B=0011=3$):
 2's complement of $B(0011) = 1101$
 0110
 + 1101

 1 0011 -> end carry discarded -> result = 0011 = +3

1.2.5 Overflow Detection

Overflow occurs when the result of signed arithmetic exceeds the representable range. It happens **only when both operands have the same sign** and the result sign differs. Detection rule: overflow = **carry-in to MSB XOR carry-out of MSB**. For n bits in 2's complement the range is $-2^{(n-1)}$ to $+2^{(n-1)} - 1$.

MCQ Trap: Adding two numbers of **opposite signs can never overflow**. A carry-out alone does **not** mean overflow in signed (2's complement) arithmetic — overflow needs $C_{in}(MSB) \neq C_{out}(MSB)$.

1.3 Data Representation

1.3.1 Signed Magnitude Representation

The **MSB is the sign bit** (0 = positive, 1 = negative); remaining bits hold the magnitude. For n bits, range = $-(2^{(n-1)} - 1)$ to $+(2^{(n-1)} - 1)$. It has **two representations of zero** (+0 and -0), which complicates arithmetic.

Representation	Range (n bits)	Zeros	Used For
Signed magnitude	$\pm(2^{(n-1)} - 1)$	Two (± 0)	Sign of mantissa
1's complement	$\pm(2^{(n-1)} - 1)$	Two (± 0)	Rarely
2's complement	$-2^{(n-1)}$ to $2^{(n-1)} - 1$	One	Modern CPUs

MCQ Trap: Only **2's complement** has a single, unique zero and one extra negative value. Signed-magnitude and 1's complement both have **+0 and -0**.

1.3.2 Decimal Fixed Point Representation

In **fixed point**, the radix point position is fixed/implied. Integers and decimals can be stored in **BCD (packed/unpacked)**. Fixed point is fast and exact for integers but has a limited, fixed range and resolution.

1.3.3 Floating Point Representation (IEEE 754)

A floating-point number has three fields: **sign (S)**, **exponent (E, stored with bias)**, and **mantissa/fraction (M)**. Value = $(-1)^S \times 1.M \times 2^{(E - \text{bias})}$. The leading **1 is implicit (hidden bit)** in normalized form.

Format	Total bits	Sign	Exponent	Mantissa	Bias
Single precision	32	1	8	23	127
Double precision	64	1	11	52	1023

IEEE 754 Single Precision (32-bit) layout:

+---+-----+				
S Exponent			Mantissa	
1 8 bits			23 bits	
+---+-----+				
31 30	23 22			0

1.3.4 Normalization and Bias

Normalization shifts the mantissa so exactly one non-zero digit (a **1** in binary) sits left of the radix point, giving form **1.xxxx**. The exponent is stored as a **biased (excess) value** so it is always non-negative: stored $E = \text{actual exponent} + \text{bias}$. Special encodings:

- $E = \text{all 0s}, M = 0 \rightarrow \pm \text{zero}$.
- $E = \text{all 0s}, M \neq 0 \rightarrow \text{denormalized (subnormal) numbers}$.
- $E = \text{all 1s}, M = 0 \rightarrow \pm \text{infinity}$.
- $E = \text{all 1s}, M \neq 0 \rightarrow \text{NaN (Not a Number)}$.

Memory Trick: "127 / 1023" are single/double biases — remember as $2^{(e-1)} - 1$ where $e =$ exponent bits (8→127, 11→1023).

MCQ Trap: Bias is added when **storing** and subtracted when **reading**. Single uses bias **127** (not 128); the +1 extra value is reserved for special encodings.

2. Codes

2.1 Binary Codes

Codes map data to binary patterns. They are **weighted** (each bit has a fixed positional weight, e.g., BCD 8421) or **non-weighted/unweighted** (Gray, Excess-3).

2.1.1 Gray Code (Reflected Binary Code)

In **Gray code** only **one bit changes** between two successive values (unit distance / cyclic code). It is **unweighted** and used in shaft encoders and K-Maps to minimize switching errors.

- **Binary → Gray:** MSB stays the same; each next Gray bit = XOR of adjacent binary bits $G_i = B_i \oplus B_{(i+1)}$.
- **Gray → Binary:** MSB stays the same; each next binary bit = XOR of previous binary bit with current Gray bit $B_i = B_{(i+1)} \oplus G_i$.

Binary 1011 → Gray:

$$\begin{aligned}
 G_3 &= B_3 &= 1 \\
 G_2 &= B_3 \oplus B_2 = 1 \oplus 0 = 1 \\
 G_1 &= B_2 \oplus B_1 = 0 \oplus 1 = 1 \\
 G_0 &= B_1 \oplus B_0 = 1 \oplus 1 = 0 \\
 \text{Gray} &= 1110
 \end{aligned}$$

2.1.2 Weighted Code — BCD / 8421

BCD (Binary Coded Decimal) encodes each decimal digit (0–9) in **4 bits** using weights **8-4-2-1**. Only **10 of 16** combinations are valid; the **6 combinations 1010–1111 are invalid**. BCD addition: if a 4-bit sum **exceeds 9** or generates a carry, **add 6 (0110)** to correct it.

2.1.3 Excess-3 Code (XS-3)

Excess-3 = BCD value + **3 (0011)**. It is **unweighted** and **self-complementing** (the 9's complement of a digit is found by simply inverting all 4 bits). This property makes XS-3 convenient for decimal subtraction.

Memory Trick: "BCD plus 3 = Excess-3." Self-complementing codes (XS-3, 2421, 84-2-1) give the **9's complement by bit inversion**.

2.1.4 ASCII and EBCDIC

Code	Bits	Characters	Origin
ASCII	7-bit (extended 8-bit)	128 (256 extended)	ANSI standard
EBCDIC	8-bit	256	IBM mainframes

In ASCII, '**A**' = **65**, '**a**' = **97**, '**0**' = **48**. The difference between an uppercase and lowercase letter is **32**.

MCQ Trap: ASCII is **7 bits** (128 symbols); EBCDIC is **8 bits** (256 symbols, IBM). Examiners swap the bit counts. Gray = **unweighted**; BCD/8421 = **weighted**; Excess-3 = **unweighted + self-complementing**.

2.2 Error Detection Codes

2.2.1 Parity Bit (Even and Odd)

A **parity bit** is an extra bit appended so the total number of 1s meets a rule. **Even parity:** total number of 1s (including parity bit) is **even**. **Odd parity:** total is **odd**. Parity detects an **odd number of bit errors (1 bit)** but **cannot correct** them and **cannot detect an even number** of errors.

2.2.2 Odd Function

The **XOR gate is the "odd function"** — its output is **1 when an odd number of inputs are 1**. XOR is used to **generate** parity; **XNOR (even function)** outputs 1 when an even number of inputs are 1. Even-parity generation uses **XOR**; odd-parity uses **XNOR**.

2.2.3 Hamming Code (Basics)

Hamming code is an **error-correcting code (ECC)** that can correct a **single-bit error**. Number of parity bits p satisfies $2^p \geq m + p + 1$ (m = data bits). Parity bits occupy positions that are **powers of 2 (1, 2, 4, 8, ...)**.

- **Hamming distance** = number of bit positions in which two codewords differ.
- To **detect** d errors, minimum distance = **$d + 1$** .
- To **correct** d errors, minimum distance = **$2d + 1$** .

Memory Trick: "Detect $d \rightarrow d+1$; Correct $d \rightarrow 2d+1$." Parity = 1-bit detect only; Hamming = single-error correct.

3. Digital Logic (Foundation)

3.1 Logic Gates

Gate	Symbol/Op	Output is 1 when	Notes
AND	$A \cdot B$	all inputs 1	—
OR	$A + B$	any input 1	—
NOT	A'	input is 0	inverter
NAND	$(A \cdot B)'$	not all inputs 1	universal
NOR	$(A + B)'$	all inputs 0	universal
XOR	$A \oplus B$	inputs differ	odd function
XNOR	$(A \oplus B)'$	inputs same	even function

MCQ Trap: **NAND** and **NOR** are the **universal gates** (any circuit can be built from either alone). **XOR** detects **difference**; **XNOR** detects **equality** (acts as a comparator).

3.2 Boolean Algebra and Laws

Key laws: **Commutative**, **Associative**, **Distributive**, **Identity** ($A + 0 = A$, $A \cdot 1 = A$), **Complement** ($A + A' = 1$, $A \cdot A' = 0$), **Idempotent** ($A + A = A$), **Absorption** ($A + AB = A$).

De Morgan's Theorems: $(A \cdot B)' = A' + B'$ and $(A + B)' = A' \cdot B'$. The **Consensus theorem:** $AB + A'C + BC = AB + A'C$.

Memory Trick: De Morgan — "**Break the bar, change the sign.**" Breaking a NOT over AND/OR flips the operator and complements each term.

3.3 Karnaugh Map (K-Map) Simplification

A **K-Map** is a graphical Boolean minimization tool. Cells are arranged in **Gray-code order** so adjacent cells differ by one bit. Group **1s** in powers of two (**1, 2, 4, 8, 16**) to get a minimal **SOP**; group **0s** for minimal **POS**. **Don't-care (X)** terms may be included to enlarge groups. Larger groups eliminate more variables.

3.4 Combinational Circuits

In **combinational circuits** output depends only on the **present inputs** (no memory).

Circuit	Function	Key Equations / Notes
Half Adder	adds 2 bits	Sum = $A \oplus B$, Carry = $A \cdot B$
Full Adder	adds 3 bits	Sum = $A \oplus B \oplus C_{in}$, Carry = $AB + C_{in}(A \oplus B)$
Half Subtractor	subtracts 2 bits	Diff = $A \oplus B$, Borrow = $A' \cdot B$
Multiplexer (MUX)	many $\rightarrow$ one	2^n inputs, n select lines, 1 output
Demultiplexer (DEMUX)	one $\rightarrow$ many	1 input, n select, 2^n outputs
Decoder	$n \rightarrow 2^n$	n inputs, 2^n outputs
Encoder	$2^n \rightarrow n$	2^n inputs, n outputs

MCQ Trap: A **MUX** is a **data selector** (many inputs, one output); a **DEMUX/decoder** is a **data distributor** (one input $\rightarrow$ many). A MUX with **n select lines** routes **2^n** inputs.

3.5 Sequential Circuits

In **sequential circuits** output depends on present inputs **and** past state (they have **memory** via feedback and a clock).

- Flip-flops:** **SR** ($S=R=1$ invalid), **JK** ($J=K=1$ toggles), **D** (output follows D, no invalid state), **T** (toggles when $T=1$).

- **Latch** = level-triggered; **Flip-flop** = edge-triggered (rising/falling clock edge).
- **Registers** store multi-bit words; **shift registers** move bits (SISO, SIPO, PISO, PIPO).
- **Counters: asynchronous/ripple** (FFs clocked in cascade, slow) vs **synchronous** (all FFs share one clock, fast).

Memory Trick: JK = "Jump (toggle) at 11", D = "Delay/Data", T = "Toggle", SR = "Set-Reset, 11 forbidden."

4. Instruction Set and Addressing

4.1 Machine Instructions

4.1.1 Machine Language

Machine language is binary code (0s and 1s) directly executed by the CPU — the only language hardware understands. An instruction has an **opcode** (operation) and one or more **operand/address** fields.

4.1.2 Instruction Formats

The number of explicit addresses defines the format:

Format	Example	Operand Location
Zero-address	PUSH, ADD	Stack (operands implicit)
One-address	ADD B	Accumulator + memory
Two-address	ADD R1, R2	Two registers/memory
Three-address	ADD R1, R2, R3	Source1, Source2, Dest

Three-address evaluation of $X = (A+B)*(C+D)$:

```
ADD R1, A, B ; R1 <- A + B
ADD R2, C, D ; R2 <- C + D
MUL X, R1, R2 ; X <- R1 * R2
```

4.1.3 Instruction Cycle (Fetch–Decode–Execute)

Every instruction passes through the cycle: **Fetch** (read instruction from memory using PC), **Decode** (interpret opcode in CU), **Execute** (perform operation in ALU), and optionally **Interrupt** check. The **PC is incremented** during fetch.

4.2 Addressing Modes

Addressing modes specify how the **effective address (EA)** of an operand is computed.

Mode	Effective Address	Use
Immediate	Operand inside instruction	constants
Direct	EA = address field	simple access
Indirect	EA = contents of address field	pointers
Register	Operand in register	fastest
Register Indirect	EA = contents of register	pointer in register
Indexed	EA = base + index register	arrays
Base Register	EA = base register + offset	relocation
Relative	EA = PC + offset	branches
Auto-increment/decrement	EA from register, then $\pm$	stacks/arrays

MCQ Trap: **Immediate** holds the operand itself (zero memory references). **Direct** = 1 memory reference; **Indirect** = 2 memory references. **Relative** mode adds the offset to the **PC**, enabling position-independent branches.

4.3 Program Control

4.3.1 Program Counter (PC)

The **Program Counter (PC)** holds the **address of the next instruction** to be fetched. It is auto-incremented after each fetch; **branch/jump/call** instructions load a new value into it.

4.3.2 Branch, Jump, Subroutine Calls

Branch/Jump alter control flow (conditional or unconditional). A **subroutine CALL** saves the **return address** (on a **stack** or in a link register) and loads the subroutine address; **RETURN** pops the saved address back into the PC.

4.3.3 Stack and Reverse Polish Notation

A **stack** is a **LIFO** structure managed by a **stack pointer (SP)**. **Reverse Polish Notation (RPN / postfix)** writes operators **after** operands (**AB+**), needs **no parentheses**, and is evaluated naturally with a stack — the basis of **zero-address** instructions.

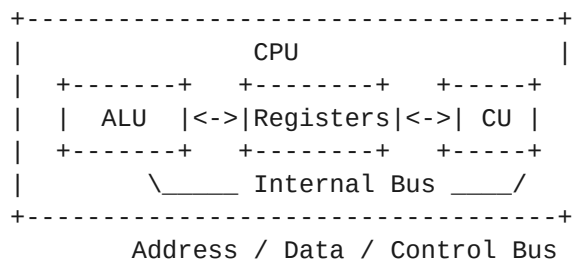
Infix $(A+B)*C \rightarrow$ Postfix (RPN): $A B + C *$
Evaluation uses PUSH/POP on a stack (zero-address machine).

Memory Trick: Postfix = operator last. Stack ops follow **LIFO**: **PUSH** adds to top, **POP** removes from top; **SP** tracks the top.

5. CPU Organization

5.1 CPU Structure

The **CPU (Central Processing Unit)** has three core parts: the **ALU** (arithmetic/logic), the **Control Unit (CU)** (generates control signals), and the **register set** (fast storage). Internally they connect through a **common bus**.



5.2 Register Organization

5.2.1 General Register Organization

A set of **general-purpose registers (GPRs)** connected via buses and a common ALU, selected by **multiplexers/decoders** and controlled by a **control word**. Modern CPUs favor many GPRs to reduce slow memory access.

5.2.2 Special Purpose Registers

Register	Full Name	Function
PC	Program Counter	address of next instruction
IR	Instruction Register	holds current instruction
MAR	Memory Address Register	address for memory access
MDR/MBR	Memory Data/Buffer Register	data to/from memory
AC	Accumulator	holds ALU results
SP	Stack Pointer	top of stack
PSW	Program Status Word	flags/condition codes

5.2.3 Stack Organization

A **stack** in the CPU (register stack or memory stack) supports **LIFO** operations for subroutine returns, expression evaluation, and interrupts. The **SP** points to the top element.

5.3 Control Unit

The **Control Unit** decodes instructions and issues timed **control signals**. The **control word** is a bit pattern where each bit enables a specific micro-operation (e.g., register load, bus enable, ALU op).

Feature	Hardwired Control	Microprogrammed Control
Implementation	Fixed logic gates/FSM	Control memory (microcode)
Speed	Faster	Slower
Flexibility	Hard to modify	Easy to modify
Cost/Design	Complex to design	Simpler, structured
Used in	RISC	CISC

5.3.1 Microprogrammed Control

In microprogrammed control, each machine instruction maps to a **microprogram** — a sequence of **microinstructions** stored in **control memory (control ROM)**. A **Control Address Register (CAR)** sequences microinstructions. **Horizontal** microcode = wide, fast, many control bits; **Vertical** microcode = narrow, encoded, slower (needs decoding).

MCQ Trap: Hardwired = fast but rigid (RISC); Microprogrammed = flexible but slower (CISC). A **microprogram** is a set of **microinstructions**; **microinstructions** contain **micro-operations**.

5.4 Arithmetic Logic Unit (ALU)

The **ALU** performs arithmetic (**add**, **subtract**, **increment**) and logic (**AND**, **OR**, **XOR**, **shift**) operations. It is built from a **parallel adder + logic circuits + a shifter**, with function selected by control lines. Status **flags** (Zero, Carry, Sign, Overflow) are set by results.

6. Pipelining and Parallelism

6.1 Instruction Pipelining Concepts

Pipelining overlaps the execution of multiple instructions by splitting the datapath into stages, like an assembly line. A classic **RISC pipeline** has stages: **IF (fetch)** → **ID (decode)** → **EX (execute)** → **MEM (memory)** → **WB (write-back)**.

Pipeline (4 instr, 5 stages) - overlap:

```

I1: IF ID EX MEM WB
I2:   IF ID EX  MEM WB
I3:     IF ID  EX  MEM WB
I4:       IF ID  EX  MEM WB

```

6.2 Pipeline Stages and Speedup

For **k** stages and **n** instructions, with each stage taking one clock cycle **tp**:

- Non-pipelined time = $n \times k \times tp$.
- Pipelined time = $(k + n - 1) \times tp$.
- **Speedup S** = $(n \times k) / (k + n - 1)$, approaching the **maximum speedup of k** as $n \rightarrow \infty$.

Memory Trick: Pipeline fills in **k cycles**, then one instruction completes **every cycle**. Total cycles = **k + (n - 1)**.

6.3 Pipeline Hazards

Hazards are situations that stall the pipeline or produce wrong results.

Hazard	Cause	Example
Structural	Resource conflict (one unit needed twice)	single memory port
Data	Dependency on a not-yet-ready result	RAW, WAR, WAW
Control	Branch/jump changes PC	conditional branch

6.3.1 Data Dependency / Data Hazards

- **RAW (Read After Write)** — *true dependency*; instruction reads a register before the prior write completes. Most common.
- **WAR (Write After Read)** — *anti-dependency*.
- **WAW (Write After Write)** — *output dependency*.

6.3.2 Hazard Handling

- **Forwarding/Bypassing:** route an ALU result directly to a waiting stage without going through registers (resolves most **RAW** hazards).
- **Stalling (bubble):** insert no-op cycles until the hazard clears.
- **Branch handling: branch prediction, delayed branch, and flushing** mitigate control hazards.

MCQ Trap: **RAW** is the **true/genuine** dependency (only one solvable by forwarding alone). **WAR** and **WAW** are **name dependencies** removable by **register renaming**. Control hazards come from **branches**, not data.

6.4 RISC vs CISC

Feature	RISC	CISC
Instruction set	Small, simple	Large, complex
Instruction length	Fixed	Variable
Addressing modes	Few	Many
Execution	Mostly 1 cycle	Multi-cycle
Memory access	Load/Store only	Many instructions access memory
Control unit	Hardwired	Microprogrammed
Registers	Many	Fewer
Examples	ARM, MIPS, RISC-V	x86, VAX

Memory Trick: **RISC = Reduced, Regular, Registers, Rapid (1-cycle), hardwired**. CISC = Complex, microcoded, memory-rich instructions.

6.5 Parallel Processing — Flynn's Classification

Flynn's taxonomy classifies architectures by instruction and data streams:

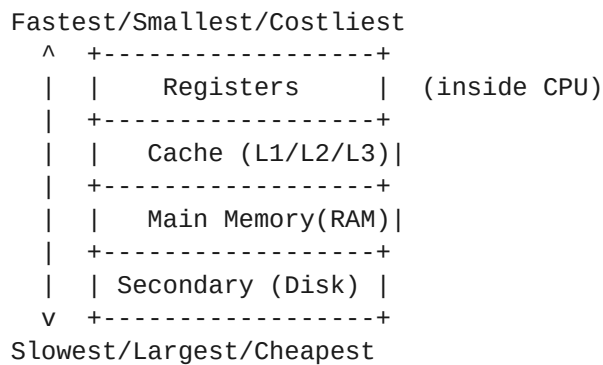
Class	Streams	Description	Example
SISD	1 instr, 1 data	Single sequential processor	classic uniprocessor
SIMD	1 instr, many data	Same op on multiple data	GPUs, vector/array procs
MISD	many instr, 1 data	Rare/theoretical	fault-tolerant pipelines
MIMD	many instr, many data	Independent processors	multicore, clusters

MCQ Trap: **SIMD** = one instruction on many data (vector/array, GPUs). **MISD** is the **rarest** (almost no real systems). Multicore CPUs are **MIMD**.

7. Memory Organization

7.1 Memory Hierarchy

The **memory hierarchy** trades speed against cost and capacity. Moving **down**: access time **increases**, cost/bit **decreases**, capacity **increases**.



7.2 Memory Types

Type	Subtype	Volatile?	Key Feature
RAM	SRAM	Yes	Flip-flops, fast, no refresh , used in cache
RAM	DRAM	Yes	Capacitor, needs refresh , dense/cheap, main memory
ROM	PROM	No	Programmed once
ROM	EPROM	No	Erased by UV light
ROM	EEPROM	No	Erased electrically (byte-wise)
ROM	Flash	No	Block-erase EEPROM (SSD/USB)

MCQ Trap: SRAM is faster and needs no refresh (used in cache); DRAM is denser/cheaper but must be refreshed (used as main memory). EPROM = UV erase; EEPROM = electrical erase.

7.3 Cache Memory

Cache is small, fast SRAM between CPU and main memory exploiting **locality of reference**: **temporal locality** (recently used data reused soon) and **spatial locality** (nearby addresses used soon).

7.3.1 Cache Mapping Techniques

Mapping	Block placement	Address fields	Replacement needed?
Direct	block → one fixed line (block mod #lines)	Tag + Line + Word	No
Associative	block → any line	Tag + Word	Yes
Set-Associative	block → any line within one set	Tag + Set + Word	Yes (within set)

Direct-mapped address breakdown:

```

+-----+-----+-----+
| TAG   | LINE  | WORD  |
+-----+-----+-----+

```

(k-way set-assoc replaces LINE with SET index)

7.3.2 Replacement Policies

When a set/cache is full, a victim block is chosen by: **LRU (Least Recently Used)**, **FIFO (First-In First-Out)**, **LFU (Least Frequently Used)**, or **Random**. Direct mapping needs **no policy** (only one possible line).

7.3.3 Write Policies

- **Write-through:** every write updates **cache and main memory** simultaneously — simple, consistent, but slower (more memory traffic).
- **Write-back:** write only to cache; main memory updated **only on eviction**, tracked by a **dirty bit** — faster, less traffic, but risks inconsistency.

7.3.4 Hit Ratio and Average Access Time

Hit ratio H = hits / total accesses; **miss ratio = 1 – H**. Average memory access time:

$$T_{avg} = H \times T_c + (1 - H) \times T_m$$

T_c = cache access time, T_m = main memory access time

Example: $H=0.9$, $T_c=10\text{ns}$, $T_m=100\text{ns}$

$$T_{avg} = 0.9 \times 10 + 0.1 \times 100 = 9 + 10 = 19 \text{ ns}$$

MCQ Trap: Direct mapping needs no replacement algorithm. A **fully associative** cache has the **most flexibility** but costly comparators. **Write-back uses a dirty bit**; write-through does not need one.

7.4 Virtual Memory and Memory Management

Virtual memory lets a program use an address space larger than physical RAM by keeping active pages in memory and the rest on disk. **Paging** uses **fixed-size pages/frames** (internal fragmentation); **segmentation** uses **variable-size logical segments** (external fragmentation). A **page table** maps virtual to physical addresses; the **TLB (Translation Lookaside Buffer)** caches recent translations.

- **Memory interleaving:** spreads consecutive addresses across multiple memory banks/modules for parallel access and higher bandwidth.
- **Associative (Content-Addressable) Memory (CAM):** searched by **content/data**, not by address — fast parallel search, used in TLBs and cache tag lookup.

MCQ Trap: **Paging** → internal fragmentation; **Segmentation** → external fragmentation. A **TLB** is an associative cache for **page-table entries**, reducing translation time.

8. Input-Output Organization

8.1 I/O Interface and Bus

An **I/O interface** bridges the CPU/memory and peripherals, handling differences in speed, data format, and signaling. Devices connect via the **I/O bus** (address, data, control lines). Addressing is **memory-mapped** (I/O shares the memory address space) or **isolated/I-O-mapped** (separate I/O address space with **IN/OUT** instructions).

8.2 Modes of Data Transfer

Mode	CPU Involvement	Mechanism	Speed
Programmed I/O	High (busy-wait/polling)	CPU checks status flag repeatedly	Slow
Interrupt-driven I/O	Medium	Device interrupts CPU when ready	Medium
DMA	Minimal	Controller transfers directly to memory	Fastest

8.2.1 Programmed I/O

The CPU continuously **polls** a status flag and transfers each word itself — simple but wastes CPU cycles in **busy waiting**.

8.2.2 Interrupt-Driven I/O

The device raises an **interrupt** when ready; the CPU suspends its task, runs an **Interrupt Service Routine (ISR)**, then resumes — no busy waiting.

8.2.3 Direct Memory Access (DMA)

A **DMA controller** transfers data **directly between an I/O device and memory**, bypassing the CPU; the CPU only initiates and is notified at the end. Modes: **cycle stealing** (one word at a time, sharing the bus) and **burst/block mode** (whole block while holding the bus). DMA improves throughput for high-speed devices like disks.

Programmed I/O	:	Device <-> CPU <-> Memory	(CPU in the path)
DMA	:	Device <-----> Memory	(CPU bypassed) (DMA controller drives the bus)

8.3 Interrupts

Interrupts are classified as **hardware vs software**, **maskable vs non-maskable (NMI)**, and **internal (traps/exceptions) vs external (device)**.

- **Priority** decides which interrupt is served first when several occur together.
- **Daisy chaining** is a **hardware priority** scheme: devices are connected in series; the one **closest to the CPU has the highest priority**.
- **Polling** is a software priority method that checks devices in a fixed order.

8.4 I/O Processor (IOP)

An **I/O Processor (IOP)** is a specialized processor that executes its own I/O programs, managing data transfer between peripherals and memory **independently of the main CPU**, freeing the CPU for computation (used in mainframes/channels).

MCQ Trap: DMA bypasses the CPU; programmed I/O keeps the CPU in the loop. In **daisy chaining**, the device **nearest the CPU has highest priority**. **NMI cannot be disabled/masked**.

9. Quick Revision and Exam Pointers

- Number-system conversions: groups of **3** (octal) and **4** (hex) bits.
- **2's complement** = invert + 1; single zero; range $-2^{(n-1)}$ to $2^{(n-1)}-1$.
- Overflow: **Cin(MSB) $\oplus$ Cout(MSB)**, only with **same-sign operands**.
- IEEE-754: single **32-bit, bias 127**; double **64-bit, bias 1023**.
- Gray = **unweighted, 1-bit change**; BCD = **8421 weighted**; Excess-3 = **BCD+3, self-complementing**.
- Parity detects **1-bit error**; Hamming **corrects** single-bit ($2^p \geq m+p+1$).
- **NAND/NOR** are universal; **XOR** = odd function.
- Pipeline speedup = $nk / (k+n-1)$, max = **k**; **RAW** is the true hazard.
- **Hardwired = RISC/fast**; Microprogrammed = **CISC/flexible**.
- Cache: **direct (no replacement)**, **associative**, **set-associative**; **write-back uses dirty bit**.
- $T_{avg} = H \cdot T_c + (1-H) \cdot T_m$. **DMA** is the fastest transfer; **daisy chain** = hardware priority.

Memory Trick: Final recall chain — "**2's-complement, 127/1023, Gray-1-bit, NAND-NOR universal, RAW true, RISC-hardwired, direct-no-replace, write-back-dirty, DMA-bypass.**"